

System Architecture

The project follows a simple layered architecture. The presentation layer renders themed dashboards, the controller layer handles authentication and business logic, and the data layer persists users, expeditions, and bookings.

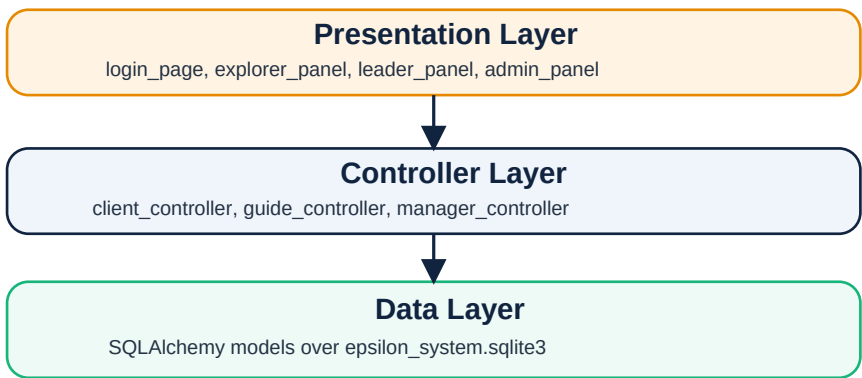


Figure 1. High-level architecture of the Flask application.

Key Functional Modules

Module	Responsibility
run_app.py	Creates the Flask app, configures SQLite, initializes SQLAlchemy, registers blueprints, and defines compatibility redirects.
client_controller.py	Handles login, registration, expedition browsing, search, and trekker booking operations.
guide_controller.py	Lets guides view assigned expeditions and update seat counts and expedition status.
manager_controller.py	Lets the manager create expeditions, verify guides, suspend users, and review the full roster.
templates/*.html	Provide role-specific dashboards with the Alpine Night visual design.

Role Workflow

- Manager: creates expeditions, verifies guide accounts, suspends trekker or guide accounts, and reviews system-wide records.
- Guide: sees only assigned expeditions and can update availability and lifecycle status.
- Trekker: searches open expeditions and reserves seats through the booking flow.